Table of contents

Network Format

Debugging target startup seque...

Debugger Request Format

Debugger Response Format

Show all...

BrightScript debug protocol

The Roku socket-based BrightScript debug protocol enables Roku app development to be tightly integrated into Visual Studio Code, Eclipse, and other Integrated Development Environments (IDEs). A tight integration helps expedite Roku app development as an IDE could be used to do the following:

- Write code using BrightScript syntax-directed editing and highlighting.
- Upload and run the app directly to the Roku media player.
- Communicate app stops and failures.
- Stop a running app.
- Inspect variables and stack traces of a stopped app.
- Step through a thread.
- Insert dynamic breakpoints.
- Resume a stopped app (if a fatal error has not occurred in the script).

This protocol may be used in an interactive debugging session. Do not use it for storing data in a static file.

Network Format

The network format of the protocol adheres to the following rules:

- Numeric values are sent in little-endian order.
- Data types are sent as a network byte stream in little-endian order.
- String values are encoded as UTF-8.

Data Type	Definition
binary32float	IEEE-754 32-bit floating-point value
binary64float	IEEE-754 64-bit floating-point value
bool	8-bit unsigned integer (0 = false; nonzero = true)
int8	8-bit signed integer
uint8	8-bit unsigned integer
int32	32-bit signed integer
uint32	32-bit unsigned integer
int64	64-bit signed integer
uint64	64-bit unsigned integer
utf8z	utf-8-encoded character stream terminated with '\0'

The notation used in this specification is similar to C++; however, the protocol describes the network data stream format—it does not define in-memory data structures.

Debugging target startup sequence

After an app is launched with a request to enable remote debugging, the firmware waits for a connection from the remote debugger client. Immediately after a connection is established, an initial handshake is then performed. The handshake consists of the following data being sent by each end of the connection:

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
struct HandshakeToDVP {    // DVP = Digital Video Player (Roku device)
    uint64 magic_number;   // 0x0067756265647362LU
};

struct HandshakeFromDVP {
    uint64 magic_number
    uint32 protocol_major_version;
    uint32 protocol_minor_version;
    uint32 protocol_patch_version;
    uint32 remaining_packet_length;
    int64  platform_revision_timestamp;
};
```

Field	Type	Description
magic_number	uint64	The Roku Brightscript debug protocol identifier, which is the following 64-bit value : `0x0067756265647362LU` . This is equal to `29120988069524322LU` or the following little-endian value: `b'bsdebug\0` .
protocol_major_version protocol_minor_version protocol_patch_version	uint32	Each Roku OS release supports only a single version of the Roku Brightscript debug protocol: **Roku OS** — **Supported Debug Protocol Version** Roku OS 14.1 — 3.3.0 Roku OS 12.0 — 3.2.0 Roku OS 11.5 — 3.1.0 Roku OS 11.0 — 3.0.0 Roku OS 9.3, 9.4, 10.0, 10.5 — 2.0.0 Roku OS 9.2 — 1.0.1 The debugger client must be updated to the protocol version number or disconnect. A change in the major version number indicates that changes that are not backwards-compatible have been made since the previous release.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Description
remaining_packet_length	uint32	The length in bytes of the remaining data, including the remaining_packet_length itself. The debugger client must read this number of bytes. As of BrightScript debug protocol 3.0.0 (Roku OS 11.0), all packets from the debugging target include a packet_length. The length is always in bytes, and includes the packet_length field, itself. This field avoids the need for changes to the major version of the protocol because it allows a debugger client to read past data it does not understand and is not critical to debugger operations. The debug target may intentionally send a packet_length longer than the actual data, with a small number of trailing padding bytes to complete the length. Clients must read the entire packet_length before expecting the next packet.
platform_revision_timestamp	int64	A platform-specific implementation timestamp (in milliseconds since epoch [1970-01-01T00:00:00.000Z]). As of BrightScript debug protocol 3.0.0 (Roku OS 11.0), a timestamp is sent to the debugger client in the initial handshake. This timestamp is platform-specific data that is included in the system software of the platform being debugged. It is changed by the platform's vendor when there is any change that affects the behavior of the debugger. The value can be used in manners similar to a build number, and is primarily used to differentiate between pre-release builds of the platform being debugged.

The behavior after the handshake has been executed, depends on the version of the BrightScript debug protocol being used:

- 2.0.0 (and later): The debug target will immediately stop on the first BrightScript statement in the script and send an ALL_THREADS_STOPPED message. The debugger client (for example, an IDE) may then set dynamic breakpoints in the target before its execution. In all cases, the debugger client must send a CONTINUE command to begin executing BrightScript code.
- 1.0.1: The debug target runs immediately after the handshake.

Debugger Request Format

Remote debugging clients can send a debugger request to the debugging target (for example, the script group) using the following packet structure for the network byte stream:

```
struct DebuggerRequest {
   uint32 packet_length;
   uint32 request_id;
   uint32 command_code;
   uint8 command_arguments;
};
```

Field	Type	Description
packet_length	uint32	The size of the packet to be sent. Example: (4 + 4 + 4 + sizeof(ARGUMENTS))
request_id	uint32	The ID of the debugger request (must be >=1). This ID is included in the debugger response.

Field	Type	Description
command_code	uint32	An enum representing the debugging command being sent, which may be one of the following values: **Code** / **Command** 1 STOP 2 CONTINUE 3 THREADS 4 STACKTRACE 5 VARIABLES 6 STEP 7 ADD_BREAKPOINTS 8 LIST_BREAKPOINTS (*As of Roku OS 11.5, this command supports both conditional and non-conditional breakpoints*) 9 REMOVE_BREAKPOINTS 10 EXECUTE 11 ADD_CONDITIONAL_BREAKPOINTS 12 SET_EXCEPTION_BREAKPOINTS 122 EXIT_CHANNEL See Debugging Commands for more information.
command_arguments (optional)	uint8	Command-specific arguments (these may not be present for some commands)

Debugger Response Format

The debugger sends responses to DebuggerRequest messages in the following format:

(https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
struct DebuggerResponse {
    uint32 packet_length;
    uint32 request_id;
    uint32 error_code;
    uint32 error_flags;
    uint8[] error_data;
    uint8 data;
};
```

Field	Type	Description
packet_length	Unit32	The length of the packet in bytes, including this field. Client must read this many bytes.
request_id	uint32	The ID of the debugger request (must be >=1). This ID is included in the debugger response.
error_code	uint32	An enum indicating the status of the request. If the debugger request was successful, a value of 0 is returned. This may be one of the following values:

Code	Status
0	OK
1	OTHER_ERR
2	UNDEFINED_COMMAND
3	CANT_CONTINUE
4	NOT_STOPPED
5	INVALID_ARGS
6	THREAD_DETACHED
7	EXECUTION_TIMEOUT

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Description

error_flags	unit32	If the value returned to the error_code field is not "OK" (error code 0), an error_flags bitmap is returned. The bitmap contains the following flags (the associated data follows the flags; their order is based on the order of the flags themselves):
		
		`enum ErrorFlags {`
		`  INVALID_VALUE_IN_PATH = 0x0001,`
		`  MISSING_KEY_IN_PATH = 0x0002`
		`};`
		
		Field / **Type** / **Summary**
		INVALID_VALUE_IN_PATH / uint32 / invalid_path_index. The index of the element in the requested path that exists, but has invalid or unknown value.
		MISSING_KEY_IN_PATH / uint32 / missing_key_index. The index of the element in path that was not found.
		
		If the error_code is set to "OK", the error_flags and error_data fields are not included in the debugger response.

error_data	uint8[]	This field is included If the value returned to the error_code field is not "OK" (error code 0) and the error_flags bitmap is not set to 0.
data	uint8	The command response returned based on the request type.

Debugger Update Format

The debugger sends an update message when a state change occurs in the application being debugged, which may or may not have been requested by the debugging client or user. DebuggerUpdate messages have a similar format as DebuggerResponse messages, except that the request_id is always 0, and it includes an update_type field, which specifies the type of update being sent.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
struct DebuggerUpdate {
    uint32 packet_length;
    uint32 request_id;
    uint32 error_code;
    uint32 update_type;
    uint8 data;
};
```

Field	Type	Description
packet_length	uint32	The length of the packet in bytes, including this field
request_id	uint32	The ID of the debugger request, which must be 0. This ID is included i debugger response. 0 is a reserved value for the request_id in DebuggerUpdate messages therefore, a debugging client may not send a DebuggerRequest with a request_id of 0.
error_code	uint32	An enum indicating the status of the request. If the debugger reques successful, a value of 0 is returned. This may be one of the following v **Code** / **Status** 0 / OK 1 / OTHER_ERR 2 / UNDEFINED_COMMAND 3 / CANT_CONTINUE 4 / NOT_STOPPED 5 / INVALID_ARGS 6 / THREAD_DETACHED 7 / EXECUTION_TIMEOUT

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Description
update_type	uint32	An enum representing the update being sent by the debugger, which one of the following values:

Code	Update	Description
0	UNDEF	
1	IO_PORT_OPENED	The remote debug client should conn the port included i data field to retrie running script's ou Only reads are allo on the I/O connect
2	ALL_THREADS_STOPPED	All threads are sto and an ALL_THREADS_ST message is sent tc debugging client. The data field inclu information on wh threads were stop

Roku (https://developer.roku.com/develop) Developers

(https://developer.roku.com/develop)

Field	Type	Description

Code	Update	Description
3	THREAD_ATTACHED	A new thread atte to execute a script all threads have al been stopped. The thread is immediat stopped and is "attached" to the debugger so that t debugger can insp the thread, its stac frames, and local variables. Additionally, when thread executes a operation, that thr detaches from the debugger tempora and a THREAD_ATTACHE message is sent to debugging client w the thread has completed its step operation and has attached to the debugger. The data field inclu information on wh threads were stop
4	BREAKPOINT_ERROR	A compilation or runtime error occu when evaluating th cond_expr of a conditional breakp
5	COMPILE_ERROR	A compilation erro occurred.
6 *Available since Roku OS 12.0*	BREAKPOINT_VERIFIED	A breakpoint has successfully been applied to an exec line of code.

Roku (https://developer.roku.com/develop) Developers

(https://developer.roku.com/develop)

Field	Type	Description
		Code / **Update** / **Description** 7 *Available since Roku OS 12.0* / PROTOCOL_ERROR / An unrecoverable e has occurred on th protocol stream. A result, the debug t is terminated. 8 *Available since Roku OS 14.1* / EXCEPTION_BREAKPOINT_ERROR / A compilation or runtime error has occurred when evaluating the cond_expr of an exception breakpo
data	uint8	The update data returned based on the update_type. This may be one following values: • If the update_type is IO_PORT_OPENED, the data field contains tl number (uint32) to which the debugging client should connect tc the script's output. • If the update_type is ALL_THREADS_STOPPED, the data field con structure named AllThreadsStoppedUpdateData. See AllThreadsStopped for more information. • If the update_type is THREAD_ATTACHED, the data field contains structure named ThreadAttachedUpdateData. See ThreadAttach more information.

AllThreadsStopped

If the update_type in a DebuggerUpdate message is set to ALL_THREADS_STOPPED, the data field contains a structure named AllThreadsStoppedUpdateData that provides the reason for the stop. The AllThreadsStoppedUpdateData structure has the following syntax:

```
struct AllThreadsStoppedUpdateData{
    int32 primary_thread_index;
    uint8 stop_reason;
    utf8z stop_reason_detail;
};
```

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Description
primary_thread_index	int32	The index of the primary thread that initiated the stop. This is -1 if the thread is unknown.
stop_reason	uint8	An enum describing why the thread was stopped. This may be one of the following values:

Value	Reason	Summary
0	UNDEFINED	Uninitialized stopReason.
1	NOT_STOPPED	Thread is running.
2	NORMAL_EXIT	Thread exited.
3	STOP_STATEMENT	Stop statement executed.
4	BREAK	Another thread in the group encountered an error or other reason outside this thread.
5	RUNTIME_ERROR	Thread stopped because of an error during execution.
6	CAUGHT_RUNTIME_ERROR	Thread stopped due to a caught runtime error. This only occurs if exception breakpoints are configured to stop on caught exceptions.

stop_reason_detail	utf8z	Provides extra details (for example, "Divide by Zero", "STOP", "BREAK")

ThreadAttached

If the update_type in a DebuggerUpdate message is set to THREAD_ATTACHED, the data field contains a structure named ThreadAttachedUpdateData that provides the reason for the stop. The ThreadAttachedUpdateData structure has the following syntax (see AllThreadsStopped for the details of each field):

```
struct ThreadAttachedUpdateData{
    int32 thread_index;
    uint8 stop_reason;
    utf8z stop_reason_detail;
}
```

BreakpointError

A BREAKPOINT_ERROR is sent if a compilation or runtime error occurs while evaluating the cond_expr of a conditional breakpoint. In this case, the update_type field in a DebuggerUpdate message is set to BREAKPOINT_ERROR, and the data field contains a structure named BreakpointErrorUpdateData that provides the reason for the error. The BreakpointErrorUpdateData structure has the following syntax:

```
struct BreakpointErrorUpdateData {
   uint32                flags;
   uint32                breakpoint_id;
   uint32                num_compile_errors;
   utf8z[num_compile_errors] compile_errors;
   uint32                num_runtime_errors;
   utf8z[num_runtime_errors] runtime_errors;
   uint32                num_other_errors;
   utf8z[num_other_errors]   other_errors;
}
```

Field	Type	Summary
flags	bool	This field is always set to 0. It is reserved for future use.
breakpoint_id	uint8	The unique ID of the breakpoint (values greater than 0 are valid; a value of 0 denotes an error).
num_compile_errors	uint32	The number of compile-time errors.
compile_errors	utf8z[num_compile_errors]	The list of compile-time errors.
num_runtime_errors	uint32	The number of runtime errors.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Summary
runtime_errors	utf8z[num_runtime_errors]	The list of runtime errors.
num_other_errors	uint32	The number of other errors (for example, permission errors).
other_errors	utf8z[num_other_errors]	The list of other errors.

CompileError

A COMPILE_ERROR is sent if a compilation error occurs. In this case, the update_type field in a DebuggerUpdate message is set to COMPILE_ERROR, and the data field contains a structure named CompileErrorUpdateData that provides the reason for the error. The CompileErrorUpdateData structure has the following syntax:

```
struct CompileErrorUpdateData {
   uint32 flags;
   utf8z  error_string;
   utf8z  file_spec;
   uint32 line_number;
   utf8z  library_name;
}
```

Field	Type	Summary
flags	bool	This field is always set to 0 (reserved for future use).
utf8z	error_string	A text message describing the compiler error.
utf8z	file_spec	A simple file path indicating where the compiler error occurred. It maps to all matching file paths in the app or its libraries "pkg:/" specifies a file in the app "lib:/<library_name>//" specifies a file in a library.
uint32	line_number	The line number where the compile error occurred.
utf8z	library_name	The name of the library where the compile error occurred.

BreakpointVerified

Available since Roku OS 12.0

Roku (https://developer.roku.com/develop) Developer
(https://developer.roku.com/develop)

A BREAKPOINT_VERIFIED message is sent when a breakpoint has successfully been applied to an executable line of code. Breakpoints may be added at any time; however, the changes may not be applied immediately if the debug target is running. In this case, the update_type field in a DebuggerUpdate message is set to BREAKPOINT_VERIFIED, and the data field contains a structure named BreakpointVerifiedUpdateData that provides the ID assigned to the verified breakpoint. The BreakpointVerifiedUpdateData structure has the following syntax:

```
struct BreakpointVerifiedUpdateData {
   uint32 flags // Reserved for future use
   uint32 num_breakpoints
   VerifiedBreakpointInfo[num_breakpoints]
}
```

```
struct VerifiedBreakpointInfo {
   uint32 breakpoint_id
}
```

Field	Type	Summary
flags	bool	This field is always set to 0. It is reserved for future use.
num_breakpoints	uint32	The number of breakpoints in the breakpoints array.
VerifiedBreakpointInfo	array	A list of verified breakpoints. Each verified breakpoint has the following syntax:

Field	Type	Summary
breakpoint_id	utf8z	The ID assigned to the breakpoint. An ID greater than 0 indicates an active breakpoint. An ID of 0 denotes that the breakpoint has an error.

ProtocolError

Available since Roku OS 12.0

A PROTOCOL_ERROR message is sent when an unrecoverable error has occurred on the protocol stream. As a result, the debug target is terminated. In this case, the update_type field in a DebuggerUpdate message is set to PROTOCOL_ERROR, and the data field contains a structure named ProtocolErrorUpdateData that provides the reason for the protocol error. The ProtocolErrorUpdateData structure has the following syntax:

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
struct ProtocolErrorUpdateData {
    uint32 flags // Reserved for future use
    uint32 protocol_error_code
}
```

```
enum ProtocolErrorCode {
    UNDEFINED = 0,
    IO_CONSOLE_FAIL = 1
}
```

Field	Type	Summary
flags	bool	This field is always set to 0. It is reserved for future use.
protocol_error_code	uint32	An enum indicating the type of protocol error that has occurred. This may be one of the following values:

Code	Error	Description
0	UNDEFINED	
1	IO_CONSOLE_FAIL	The connection on the I/O port failed (this typically means that the client did not connect within the timeout).

ExceptionBreakpointError

An EXCEPTION_BREAKPOINT_ERROR is sent if a compilation or runtime error occurs while evaluating the cond_expr of an exception breakpoint. In this case, the update_type field in a DebuggerUpdate message is set to EXCEPTION_BREAKPOINT_ERROR, and the data field contains a structure named ExceptionBreakpointErrorUpdateData that provides the reason for the error. The ExceptionBreakpointErrorUpdateData structure has the following syntax:

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
struct ExceptionBreakpointErrorUpdateData {
    uint32                  flags;
    uint32                  filter_id;
    uint32                  num_compile_errors;
    utf8z[num_compile_errors] compile_errors;
    uint32                  num_runtime_errors;
    utf8z[num_runtime_errors] runtime_errors;
    uint32                  num_other_errors;
    utf8z[num_other_errors]   other_errors;
    uint32                  line_number;
    utf8z                   file_path;
}
```

Field	Type	Summary
flags	bool	This field is always set to 0. It is reserved for future use.
filter_id	uint32	The filter ID of the breakpoint **Value** / **Filter ID** / **Description** 1 / CAUGHT / Stop on all caught exceptions. 2 / UNCAUGHT / Stop on all uncaught exceptions.
num_compile_errors	uint32	The number of compile-time errors.
compile_errors	utf8z[num_compile_errors]	The list of compile-time errors.
num_runtime_errors	uint32	The number of runtime errors.
runtime_errors	utf8z[num_runtime_errors]	The list of runtime errors.
num_other_errors	uint32	The number of other errors (for example, permission errors).
other_errors	utf8z[num_other_errors]	The list of other errors.
line_number	uint32	The line number where the condition failed to evaluate.
file_path	utf8z	the file path where the condition failed to evaluate.

Debugging Commands

The BrightScript debugger supports the following debug commands:

Debug Command	Description	Access Scope
STOP	Stop all threads in application. Enter into debugger. Individual threads can not be stopped/started.	Application is running
CONTINUE	Exit from debugger and continue execution of all threads.	Debugger is active. All threads are stopped
THREADS	Application threads info	Debugger is active. All threads are stopped.
STACKTRACE	Get the stack trace of a specific thread.	Debugger is active. All threads are stopped.
VARIABLES	Listing of variables accessible from selected thread and stack frame.	Debugger is active, all thread
STEP	Execute one step on a specified thread.	Debugger is active. All threads are stopped. As of Roku OS 14.6, you can use the STEP command to step over and out of SceneGraph observer callbacks and functions called via CallFunc (/docs/developer-program/core-concepts/handling-application-events.md#functional-fields).
ADD_BREAKPOINTS	Add a dynamic breakpoint.	Debugger is active. Application is active (may be stopped or running).

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Debug Command	Description	Access Scope
LIST_BREAKPOINTS	Lists existing dynamic and conditional breakpoints and their status.	Debugger is active. All threads in script group are stopped.
REMOVE_BREAKPOINTS	Removes dynamic breakpoints.	Debugger is active. All threads in script group are stopped.
EXECUTE	Executes code in a specific stack frame.	Debugger is active
ADD_CONDITIONAL_BREAKPOINTS	Adds a conditional breakpoint.	Debugger is active. App is active (may be stopped). The app or script must be stopped for an ADD_CONDITIONAL_BREAKPOINTS request to be accepted.
SET_EXCEPTION_BREAKPOINTS	Configure exception breakpoints.	Debugger is active

ThreadsResponse

The ThreadsResponse struct has the following syntax:

```
struct ThreadsResponse{
   uint32 threads_count;
   ThreadInfo[] threads;
};
```

Field	Type	Summary
threads_count	uint32	The number of threads in the response.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Summary
threads	ThreadInfo[]	An array of ThreadInfo structs. A ThreadInfo struct has the follc

```
struct ThreadInfo{
   uint8 flags;
   uint8 stop_reason;
   utf8z stop_reason_detail;
   uint32 line_number;
   utf8z function_name;
   utf8z file_path;
   utf8z code_snippet;
};
```

Field	Type	Summary
flags	uint8	Contains a ThreadInfoFlags er indicates whether this thread or failure. IS_PRIMARY is set t This enum uses a bitwise mas into 8 bits.

Field	Type	Summary
stop_reason	uint32	An enum describing why the t This may be one of the followi **Value** / **Reason** 0 — UNDEFINED 1 — NOT_STOPPED 2 — NORMAL_EXIT 3 — STOP_STATEMENT 4 — BREAK 5 — RUNTIME_ERROR The stop_reason is an 8-bit other objects in this protoc sent in the ThreadsRespons for historical purposes.
stop_reason_detail	utf8z	Provides extra details about t "Divide by Zero", "STOP", "BREA
line_number	uint32	The line number where the st
function_name	utf8z	The function where the stop c
file_path	utf8z	The file where the stop or failu

Roku (https://developer.roku.com/develop) Developers

(https://developer.roku.com/develop)

Field	Type	Summary

Field	Type	Summary
code_snippet	utf8z	The code causing the stop or

StackTraceResponse

The StackTraceReponse struct has the following syntax:

```
struct StackTraceResponse{
   uint32 stack_size;
   StackEntry[] entries;
};
```

Field	Type	Summary
stack_size	uint32	The number of stack entries in the entires array.

entries	StackEntry[]	An array of StrackEntry structs. entries[0] contains the last function called; entries[stack_size-1] contains the first function called. Debugging clients may reverse the entries to match developer expectations.
		
		A StrackEntry struct has the following syntax:
		
		`struct StackEntry{`
		`  uint32 line_number;`
		`  utf8z function_name;`
		`  utf8z file_name;`
		`};`

Field	Type	Summary
line_number	uint32	The line number where the stop or failure occurred.
function_name	utf8z	The function where the stop or failure occurred.
file_name	utf8z	The file where the stop or failure occurred.

Variables arguments

Argument	Type	Summary
variable_request_flags	uint8	Contains one the following VariableRequestFlags enums: ${variable-request-flags-table} This enum uses a bitwise mask that enables it to fit into 8 bits.
thread_index	uint32	The index of the thread containing the variable.
stack_frame_index	uint32	The index of the frame returned from the STACKTRACE command. The 0 index contains the first function called; nframes-1 contains the last. This indexing does not match the order of the frames returned from the STACKTRACE command
variable_path_len	uint32	The number of variable_path entries. If this is set to 0, the variables that are accessible from the specified stack frame are returned.
variable_path_entries	utf8z[]	A set of one or more path entries to the variable to be inspected. For example, `m.top.myarray[6]` can be accessed with `["m","top","myarray","6"]`. If no path is specified, the variables accessible from the specified stack frame are returned.
path_force_case_insensitive	bool	Forces a case-insensitive lookup of the corresponding path entry when enabled. Enabling this flag also requires the variable_request_flags argument to be set to CASE_SENSITIVITY_OPTIONS. This is useful for debugging scripts using "." notation for associative arrays, which is always case insensitive for all object types.
path_is_virtual *Available since Roku OS 14.1*	bool[]	Indicates that the path entry is virtual and does not correspond to a real variable. Enabling this flag also requires variable_request_flags to be set to VIRTUAL_PATH_INCLUDED.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Value	Summary
GET_CHILD_KEYS	Indicates whether the VARIABLES response includes the child keys for container types like lists and associative arrays. If this is set to true (0x01), the VARIABLES response include the child keys.
CASE_SENSITIVITY_OPTIONS	Enables the client application to send path_force_case_insensitive data
GET_VIRTUAL_KEYS	Indicates whether the VARIABLES response includes virtual keys for the requested paths. See Virtual Variables
VIRTUAL_PATH_INCLUDED *Available since Roku OS 14.1*	Enable the client application to sent path_is_virtual data.

VariablesResponse

The VariablesResponse struct has the following syntax:

```
struct VariablesResponse{
   uint32 num_variables;
   VariableInfo[] variables;
};
```

Field	Type	Summary
num_variables	uint32	The number of variables in the variables array.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Summary

variables	VariableInfo[]	An array of VariableInfo structs. A VariableInfo struct has the following syntax:
		
		`struct VariableInfo{`
		`  uint8 flags;`
		`  uint8 variable_type;`
		`  utf8z name;`
		`  uint32 ref_count;`
		`  uint8 key_type;`
		`  uint32 element_count;`
		`  void* value;`
		`};`

Field	Type	Summary

Argument	Type	Summary
flags	uint8	The flags that determine which fields are included in the VariableInfo struct. This field is always listed, and it may be set to one of the following values: • 0x01 = IS_CHILD_KEY (the value is a child of the requested variable; for example, an element of an array or field of an AA) • 0x02 = IS_CONST (value is constant) • 0x04 = IS_CONTAINER (the referenced value is a container; for example, a list or array) • 0x08 = IS_NAME_HERE (the name is included in this VariableInfo) • 0x10 = IS_REF_COUNTED (the value is reference-counted). • 0x20 = IS_VALUE_HERE (the value is included in this VariableInfo) • 0x40 = IS_KEYS_CASE_SENSITIVE (the value is a container with case-sensitive keys) • 0x80 = IS_VIRTUAL (the value is virtual and does not correspond to a real variable)

Field	Type	Summary

Argument	Type	Summary
variable_type	Uint8	Contains an enum, ValueType, which indicates the type of variable/value. This field is always listed, and it may be set to one of the following values: • 1 = AA • 2 = ARRAY • 3 = BOOLEAN • 4 = DOUBLE • 5 = FLOAT • 6 = FUNCTION • 7 = INTEGER • 8 = INTERFACE • 9 = INVALID = 9 (literal BrightScript Invalid value) • 10 = LIST • 11 = LONG_INTEGER • 12 = OBJECT • 13 = STRING • 14 = SUBROUTINE • 15 = SUBTYPED_OBJECT • 16 = UNINITIALIZED (the variable exists, but it has no value or type) • 17 = UNKNOWN (the variable is valid, but its type is unknown yet not undefined)
name	utf8z	The variable name. The field is only listed if the flags field includes the IS_NAME_HERE flag.
ref_count	uint32	The number of references this variable has. The field is only listed if the flags field includes the IS_REF_COUNTED flag.

Field	Type	Summary

Argument	Type	Summary
key_type	uint8	The type of keys in the container. The field is only listed if the flags field includes the IS_CONTAINER flag. This field contains an enum, ValueType, which indicates the type of variable/value (see the variable_type field for more information).
element_count	uint32	The number of elements in the container. The field is only listed if the flags field includes the IS_CONTAINER flag
value	void*	A type-dependent value based on the variable_type field. It is not present for all types.

The data segment of a VariableInfo byte stream contains one of the following data sets :

- Value_AA {no data} (use GET_CHILD_KEYS in request to get contents)
- Value_Array {no data}
- Value_Boolean {uint8 value;} // 0 = false, otherwise true
- Value_Double {binary64float value;}
- Value_Float {binary32float value;}
- Value_Function {uint8 function_name;}
- Value_Integer {int32 value;}
- Value_Interface {utf8z interface_name;}
- Value_Invalid {no data}
- Value_List {no data}
- Value_LongInteger {int64 value;}
- Value_Object {utf8z class_name;}
- Value_String {utf8z value;}
- Value_Subroutine {utf8z subroutine_name;}
- Value_SubtypedObject {utf8z class_name; utf8z subtype_name;}
- Value_Uninitialized {no data}

Field	Type	Summary
		• Value_Unknown {no data}

Step arguments

Argument	Type	Summary
thread_index	uint32	The index of the thread to step through.
step_type	uint8	Contains an a StepType enum, indicating the type of step action to be executed. This may be on the following values: • 0 = STEP_TYPE_NONE • 1 = STEP_TYPE_LINE • 2 = STEP_TYPE_OUT • 3 = STEP_TYPE_OVER

Execute arguments

Argument	Type	Summary
thread_idx	uint32	The index of the thread to be executed.
stack_id	uint32	The stack frame containing the thread to be executed.
source_code	utf8z	The source code to be executed.

ExecuteResponseData

- Success: ErrorCode::OK. The code snippet was legal BrightScript and no compile-time errors occurred. However, the code itself may still generate a runtime error. For example, the code snippet "x = 5 / 0" will compile but generate a "divide by zero" runtime error. This error would be sent as text to the output stream of the debugging connection.

  If the error_code is `ErrorCode::OK`, the following fields are also included:

Field	Type	Summary
execute_success	bool	Indicates whether the code ran and completed without errors (true).

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Summary
runtime_stop_code	uint8	A StopReason enum.
num_compile_errors	uint32	The number of compile-time errors.
compile_errors	utf8z[num_compile_errors]	The list of compile-time errors.
num_runtime_errors	uint32	The number of runtime errors.
runtime_errors	utf8z[num_runtime_errors]	The list of runtime errors.
num_other_errors	uint32	The number of other errors (for example, permission errors).
other_errors	utf8z[num_other_errors]	The list of other errors.

- Failure: other ErrorCode. No additional fields are included.

Dynamic Breakpoints

Dynamic breakpoints enable developers to navigate through the app, inspect its state, and view its execution flow when a specific runtime conditions occurs. The debug protocol includes commands for adding, listing, and removing breakpoints.

AddBreakpointsRequestArgs

The AddBreakpointsRequestArgs struct has the following syntax:

```
struct AddBreakpointsRequestArgs {
   uint32 num_breakpoints;
   BreakpointSpec[] breakpoints;
};
```

Field	Type	Summary
num_breakpoints	uint32	The number of breakpoints in the breakpoints array.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Summary

breakpoints	BreakpointSpec[]	An array of BreakpointSpec structs. A BreakpointSpec struct has the following syntax:
		
		`struct BreakpointSpec {`
		`  utf8z file_spec;`
		`  uint32 line_number;`
		`  uint32 ignore_count;`
		`};`
		
		Argument / **Type** / **Summary**
		file_spec / utf8z / The simple path of the source file where the breakpoint is to be inserted.
		
		"pkg:/" specifies a file in the app
		
		"lib:/<library_name>/" specifies a file in a library.
		
		line_number / uint32 / The line number in the app code where the breakpoint is to be executed.
		
		ignore_count / uint32 / The number of times to ignore the breakpoint condition before executing the breakpoint. This number is decremented each time the app reaches the breakpoint.

AddBreakpointsResponseData

The AddBreakpointsResponseData struct has the following syntax:

```
struct AddBreakpointsResponseData {
  uint32 num_breakpoints;
  BreakpointInfo[] breakpoint_responses;
};
```

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Summary
num_breakpoints	uint32	The number of breakpoints in the breakpoint_respo
breakpoint_responses	BreakpointInfo[]	An array of BreakpointInfo structs. A BreakpointInfo following syntax:

```
struct BreakpointInfo {
   uint32 breakpoint_id;
   uint32 error_code;
   uint32 ignore_count;
};
```

Argument	Type	Summary
breakpoint_id	utf8z	The ID assigned to the b greater than 0 indicates breakpoint. An ID of 0 d breakpoint has an error.
error_code	uint32	Indicates whether the b successfully added. This following values:
ignore_count	uint32	The number of times to breakpoint condition be breakpoint. This numbe each time the app reacl This argument is only pr breakpoint_id is valid.

Code	Status
0	OK
5	INVALID_ARG

ListBreakpointsResponseData

The ListBreakpointsResponseData struct has the following syntax:


```
struct ListBreakpointsResponseData {
   uint32 num_breakpoints;
   BreakpointInfo[] breakpoints;
};
```

Field	Type	Summary
num_breakpoints	uint32	The number of breakpoints in the breakpoints array.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Summary
breakpoints	BreakpointInfo[]	An array of BreakpointInfo structs. A BreakpointInfo stru following syntax:

```
struct BreakpointInfo {
   uint32 breakpoint_id;
   uint32 error_code;
   uint32 ignore_count;
};
```

Argument	Type	Summary
breakpoint_id	utf8z	The ID assigned to the break greater than 0 indicates an a breakpoint. An ID of 0 denot breakpoint has an error.
error_code	uint32	Indicates whether the break successfully returned. This n the following values:

Code	Status
0	OK
5	INVALID_ARGS

Argument	Type	Summary
ignore_count	uint32	Current state, decreases as l executed. This argument is c the breakpoint_id is valid.

RemoveBreakpointsRequestArgs

The RemoveBreakpointsRequestArgs struct has the following syntax:

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

```
struct RemoveBreakpointsRequestArgs {
   uint32 num_breakpoints;
   uint32[] breakpoint_ids;
};
```

Field	Type	Summary
num_breakpoints	uint32	The number of breakpoints in the breakpoint_ids array.
breakpoint_ids	uint32[]	An array of breakpoint IDs representing the breakpoints to be removed.

RemoveBreakpointsResponseData

The RemoveBreakpointsResponseData struct has the following syntax:

```
struct RemoveBreakpointsResponseData {
   uint32 num_breakpoints;
   BreakpointInfo[] breakpoint_infos;
};
```

Field	Type	Summary
num_breakpoints	uint32	The number of breakpoints in the breakpoint_infos array

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Summary
breakpoint_infos	BreakpointInfo[]	An array of BreakpointInfo structs. A BreakpointInfo stru following syntax:

```
struct BreakpointInfo {
   uint32 breakpoint_id;
   uint32 error_code;
   uint32 ignore_count;
};
```

Argument	Type	Summary
breakpoint_id	utf8z	The ID assigned to the break greater than 0 indicates an a breakpoint. An ID of 0 denot breakpoint has an error.
error_code	uint32	Indicates whether the break successfully removed. This n following values:

Code	Status
0	OK
5	INVALID_ARGS

Argument	Type	Summary
ignore_count	uint32	Current state, decreases as l executed. This argument is c the breakpoint_id is valid.

Conditional Breakpoints

Conditional breakpoints enable developers to break inside a code block when a defined expression evaluates to true. Clients must use the ADD_CONDITIONAL_BREAKPOINTS debug command to add breakpoints that have conditional expressions (the ADD_BREAKPOINTS command must be used to add breakpoints without conditional expressions).

Use the LIST_BREAKPOINTS debugging command to get the existing conditional breakpoints and their status.

AddConditonalBreakpointsRequestArgs

The AddConditonalBreakpointsRequestArgs struct has the following syntax:

```
struct AddBreakpointsRequestArgs {
   uint32 flags;
   uint32 num_breakpoints;
   ConditionalBreakpointSpec[] breakpoints;
};
```

Field	Type	Summary
flags	uint32	This field is always set to 0 (reserved for fut
num_breakpoints	uint32	The number of breakpoints in the breakpoin

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Summary
breakpoints	ConditionalBreakpointSpec[]	An array of ConditonalBreakpointSpec struc ConditonalBreakpointSpec struct has the fo

```
struct CondtionalBreakpointSpec {
   utf8z file_spec;
   uint32 line_number;
   uint32 ignore_count;
   utf8z cond_expr;  //available since Debug P
};
```

Argument	Type	Summary
file_spec	utf8z	The path of the s where the condi breakpoint is to "pkg://<filepath> file in the app "lib:/<library_nar specifies a file in
line_number	uint32	The line number code where the to be executed.
ignore_count	uint32	The number of t the breakpoint c before executing breakpoint. This decremented ea app reaches the cond_expr is spe ignore_count is c it evaluates to tr
cond_expr	utf8z	BrightScript cod evaluates to a bo The cond_expr is executed in the c the breakpoint is cond_expr is spe ignore_count is c updated if this e true.

AddConditonalBreakpointsResponseData

The AddConditonalBreakpointsResponseData struct has the following syntax:


```
struct AddConditonalBreakpointsResponseData {
   uint32 num_breakpoints;
   ConditionalBreakpointInfo[] breakpoint_responses;
};
```

Field	Type	Summary
num_breakpoints	uint32	The number of breakpoints in the breakp

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Summary
breakpoint_responses	ConditonalBreakpointInfo[]	An array of ConditonalBreakpointInfo str ConditonalBreakpointInfo struct has the

```
struct ConditionalBreakpointInfo {
   uint32 breakpoint_id;
   uint32 error_code;
   uint32 ignore_count;
};
```

Argument	Type	Summary
breakpoint_id	utf8z	The ID assigr greater than breakpoint. A breakpoint h
error_code	uint32	Indicates wh successfully following val
ignore_count	uint32	The number breakpoint c breakpoint. T each time th This argume breakpoint_i

Code	St
0	Ok
5	IN

Exception Breakpoints

Available since Roku OS 14.1

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Exception breakpoints enable developers to pause the debugger whenever a runtime error is encountered or an exception is thrown. Unlike other breakpoints, exception breakpoints do not have an associated source file and line number, and they can not be listed or removed using LIST_BREAKPOINTS or REMOVE_BREAKPOINTS. Clients must use SET_EXCEPTION_BREAKPOINTS to set or clear the active exception breakpoints.

SetExceptionBreakpointsRequestArgs

The SetExceptionBreakpointsRequestArgs struct has the following syntax:

```
struct SetExceptionBreakpointsRequestArgs {
   uint32 num_breakpoints;
   ExceptionBreakpointSpec[] breakpoints;
};
```

Field	Type	Summary
num_breakpoints	uint32	The number of breakpoints in the breakpoints

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Summary
breakpoints	ExceptionBreakpointSpec[]	An array of ExceptionBreakpointSpec structs. ExceptionBreakpointSpec struct has the follo

```
struct ExceptionBreakpointSpec {
   uint32 filter_id;
   utf8z cond_expr;
};
```

Argument	Type	Summary
filter_id	uint32	The type of exceptio trigger a stop. Note bitfield. To specify m client must send mu ExceptionBreakpoint
cond_expr	utf8z	BrightScript code th boolean value. The c compiled and execut where the breakpoin

Value	Filter ID
1	CAUGHT
2	UNCAUGH

SetExceptionBreakpointsResponseData

The SetExceptionBreakpointsResponseData struct has the following syntax:

```
struct SetExceptionBreakpointsResponseData {
   uint32 num_breakpoints;
   ExceptionBreakpointInfo[] breakpoint_responses;
};
```

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Field	Type	Summary
num_breakpoints	uint32	The number of breakpoints in the breakpo
breakpoint_responses	ExceptionBreakpointInfo[]	An array of ExceptionBreakpointInfo struc struct has the following syntax:

```
struct ExceptionBreakpointInfo {
   uint32 filter_id;
   uint32 error_code;
};
```

Argument	Type	Summary
filter_id	uint32	The filter_id of tl filters-table}
error_code	uint32	Indicates whethe successfully add following values:

Code	Status
0	OK
5	INVALI

Virtual Variables

Available since Roku OS 14.1

Virtual variables are values that can be retrieved with a VARIABLES request but do not correspond to actual variables (for example, the length of a container). By convention, variables start with a `$` character. The following virtual variables are supported:

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Object type	Name	Type	Description
roSGNode	$children	roArray	The SceneGraph children of the given node. This is equivalent to calling `node.getChildren(-1, 0)`.
roSGNode	$parent	roSGNode	The SceneGraph parent of the given node. This is equivalent to calling `node.getParent()`.
roSGNode	$threadinfo	roAssociativeArray	The threadInfo of the given node. This is equivalent to calling `node.threadInfo()`.
Container types	$count	Integer	The number of elements in the container. This is equivalent to calling `var.Count()`.

Virtual variables are only returned if both GET_VIRTUAL_KEYS and GET_CHILD_KEYS are set in the VARIABLES request. Variable paths may include multiple virtual keys. For example, to get the first grandchild node's thread info, the client can send a request with the following path: `node.$children.0.$children.0.$threadinfo`.

Sample remote debugger

You can download the Roku Remote Debugger (https://github.com/rokudev/remote-debugger), which is a Python-based sample command-line remote debugger for testing and debugging Roku apps under development. The Roku Remote Debugger (rokudebug.py) provides the same functionality as the BrightScript debug console (/docs/developer-program/debugging/debugging-channels.md#brightscript-console-port-8085-commands); however, it demonstrates how the BrightScript network debug protocol could be used to integrate a debug tool into an IDE.

To run the Roku Remote Debugger, follow these steps:

1. Verify that you have Python 3.5.3 (or greater) installed on your machine.
2. Create a ZIP file (/docs/developer-program/getting-started/hello-world.md#compressing-the-contents-of-the-hello-world-directory) containing the development app to be tested. You can also download sample apps (https://github.com/rokudev/samples) to test with the debugger.
3. Sideload an app by entering the following command in a terminal or command prompt:

   `python rokudebug.py --targetip <Roku device IP address> --targetpass <Roku device webserver`

   The following example demonstrates a command for running the debugger

   `python3 rokudebug.py --targetip 192.168.1.10 --targetpass abcd VideoListExample/Archive.zip`

4. Enter help to view a list of the available debug commands, which are as follows:

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Command	Abbreviation	Description
addbreak	break, ab	Adds a breakpoint
backtrace	bt	Print stack backtrace of selected thread.
continue	c	Continue all threads.
down	d	Move one frame down the function call stack.
help	h	Print the available commands.
list	l	List the currently running function.
listbreak	Lb	List all breakpoints
out	o	Step out of the current function
over	v	Step over one program statement
print *var*		Print the value of a specific variable.
rmbreak *breakpointid*	rb	Clears the specified breakpoint
quit	q	Quit the Roku Remote Debugger and exit the app.
status		Show the status of the Roku Remote Debugger.
step	S, t	Step one program statement
stop		Stop all threads.
thread	th	Inspect a thread.
threads	ths	Show all threads.
up	u	Move one frame up the function call stack.
vars	v	Show the variables in the current scope.

[BETA] Visual Studio Code extension

You can download (https://github.com/rokudev/debug-protocol-vscode-ext-beta) the beta version of the Visual Studio Code extension for the Roku BrightScript debug protocol. After extracting and installing the extension, you can use it for debugging Roku apps in Visual Studio.

Demo video

The following video demonstrates the Roku Remote Debugger (https://github.com/rokudev/remote-debugger), and it shows how the BrightScript network debug protocol could be used in an integration with an IDE such as Visual Studio Code.

0:00 / 12:20

Change log

- 12-28-2024: Roku Remote debugger 3.3.0 release.
  - DebuggerRequest messages now support a SET_EXCEPTION_BREAKPOINTS debugging command (code 12).
  - DebuggerUpdate messages include the EXCEPTION_BREAKPOINT_ERROR (updat_type code 8) command.
  - The Variables debug command now supports requesting "virtual" variables using the GET_VIRTUAL_KEYS flag and VIRTUAL_PATH_INCLUDED keys.
- 03-06-2023: Roku Remote debugger 3.2.0 release.

- Support added for sending ADD_CONDITIONAL_BREAKPOINTS requests while the script is running.
- DebuggerUpdate messages now include BREAKPOINT_VERIFIED (update_type code 6) and PROTOCOL_ERROR (update_type code 7) commands.

- 09-12-2022: Roku Remote debugger 3.1.0 release.
  - DebuggerRequest messages now support an ADD_CONDITIONAL_BREAKPOINTS debugging command (code 11). The LIST_BREAKPOINTS debugging command (code 7) now supports both conditional and non-conditional breakpoints.
  - DebuggerResponse messages now include error_flags and error_data fields if the value returned to the error_code field is not "OK" (error code 0).
  - DebuggerUpdate messages include the following new update_type codes:
    - BREAKPOINT_ERROR (update_type code 4). A compilation or runtime error occurred when evaluating the cond_expr of a conditional breakpoint
    - COMPILE_ERROR (update_type code 5). A compilation error occurred.
  - The Variables debug command now supports a new path_force_case_insensitive flag that forces a case-insensitive lookup of the corresponding path entry when enabled. Enabling this flag also requires the VariablesRequestFlag argument to be set to the new "CASE_SENSITIVITY_OPTIONS" value. This is useful for debugging scripts using "." notation for associative arrays, which is always case insensitive for all object types.
- 03-22-2022: Roku Remote debugger 3.0.0 release.
  - The HandshakeFromDVP object, which is sent by a Roku device to a debugger client as part of the initial handshake, includes a new platform_revision_timestamp field that is primarily used to identify the Roku OS version of the device being used for debugging.
  - DebuggerResponse messages include a new packet_length field that enables a debugger client to read past non-essential data that it does not understand.
  - DebuggerUpdate messages include the following new THREAD_DETACHED (error code 6) and EXECUTION_TIMEOUT (error code 7) error_code status and codes:
- 08-14-2020: Beta release of Visual Studio Code extension. Updated debug command table.
- 03-29-2020: Roku Remote debugger 2.0.0 release. Added breakpoint and step commands.
- 11-09-2019: Roku Remote debugger 1.0.1 release.

Previous

Terms and Conditions

B2B Privacy Notice (https://docs.roku.com/published/b2bprivacypolicy/en/us)

Developer Site (https://developer.roku.com)

About ads & cookies (https://docs.roku.com/published/cookiepolicy)

My Privacy Choices
(https://privacy.roku.com/ccpa)
Developer Terms & Agreements (https://developer.roku.com/docs/features/legal/developer-terms.md)

CA B2B Privacy Notice

© 2008-2026 Roku, Inc. (https://www.roku.com)